Secure-by-Default Guardrails for MCP-Based Tool Use in Multi-Modal LLM Agents

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

***Abstract*—As autonomous AI agents transition from passive assistants to proactive system operators via the Model Context Protocol (MCP), the structural prevention of unauthorized operations becomes the first line of defense. This paper, representing Phase 1: Guardrails (Pre-execution Security) of a comprehensive security trilogy, introduces a multi-layered containment architecture for LLM-based command-line interfaces. We propose a "No-Shell" execution model that eliminates shell-injection vulnerabilities by substituting raw command execution with sandboxed Python environments. Our framework, implemented in the `llm-cli` system, integrates static code analysis (AST inspection), Linux-based process isolation (Bubblewrap), and a high-performance entropy-based secret detection mechanism. By enforcing a mandatory "Intent Explanation" protocol within a Human-in-the-Loop (HITL) workflow, we demonstrate how foundational guardrails can establish a robust security baseline for autonomous tool use without compromising agentic flexibility.**

***Index Terms*—AI Agents, Guardrails, Pre-execution Security, Model Context Protocol, Sandboxing, Human-in-the-Loop, Structural Immunity.**

I. Introduction

The evolution of Large Language Models (LLMs) from text-generators to autonomous agents has introduced a new class of security vulnerabilities. Modern agents utilize the Model Context Protocol (MCP) [1] to interact with the physical and digital world, executing code, managing files, and accessing remote APIs. This "Agency" capability creates a significant gap between the AI's intent and the system's safety. Granting an AI agent raw shell access is equivalent to providing a persistent, non-deterministic remote shell to an entity susceptible to prompt injection.

We identify this challenge as the *Structural Vulnerability of Autonomous Systems*. Traditional security assumes a human user with a stable identity and intent. AI agents, however, operate in a high-entropy reasoning space where a single adversarial prompt can "jailbreak" the agent's logic, causing it to generate harmful system commands.

This paper is the foundational installment of a technical trilogy dedicated to agent security:

- **Phase 1: Guardrails (Pre-execution Security).** The current work, focusing on structural immunity and containment to prevent unauthorized actions at the point of inception.
- **Phase 2: Zero Trust (Execution Security) [2].** Focuses on spatial security and behavioral integrity monitoring via Mamba-based Sentinels.
- **Phase 3: Post-Quantum (Temporal Security) [3].** Focuses on long-term integrity and quantum-resistant governance.

In Phase 1, we advocate for a **Secure-by-Default Guardrail** architecture. Our contribution is a unified security framework that treats the LLM's output as an inherently untrusted payload, subjecting it to multi-layered static and dynamic isolation before execution.

II. Related Work

Existing autonomous frameworks like AutoGPT or BabyAGI often rely on simple string-matching or permissive shell access, making them vulnerable to "Jailbreak" attacks where the agent is tricked into executing malicious commands (e.g., `rm -rf /`). While tools like *Guardrails AI* provide schema validation for JSON outputs, they lack low-level system containment and operating system integration.

Recent research into LLM security has focused on adversarial training and "alignment." However, alignment is probabilistic and can be bypassed by sophisticated prompts. Our approach shifts the focus from *behavioral alignment* to *structural containment*, ensuring that even a compromised or misaligned LLM cannot breach the host system's integrity.

III. Threat Model

We define three primary threat vectors for autonomous LLM agents:

1) **Prompt Injection (Direct/Indirect):** An attacker manipulates the agent's context (e.g., via a malicious website the agent is reading) to execute unauthorized system commands.
2) **Secret Exfiltration:** The agent accidentally leaks API keys, passwords, or sensitive PII (Personally Identifiable Information) to a 3rd-party LLM provider during tool calls or reasoning.
3) **Unauthorized Resource Consumption (DoS):** Maliciously generated code causing CPU exhaustion, memory leaks, or disk space depletion to render the host system unusable.

IV. Architecture of Autonomous Guardrails

Our architecture (Fig. 1) establishes five distinct layers of defense that must be satisfied before any tool-call impacts the host.

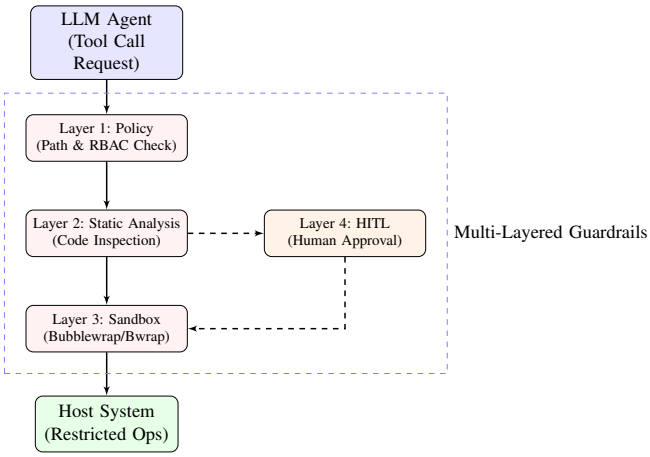

Fig. 1. Multi-layered guardrail architecture for structural containment.

A. Layer 1: The "No-Shell" Execution Model

Raw shell execution (`/bin/sh -c`) is structurally vulnerable to injection because it treats data and commands as a single string to be parsed. `llm-cli` adopts a "No-Shell" model where all system operations are performed via a specialized `execute_python` tool.

By executing Python code directly using `subprocess.run(args, shell=False)`, we eliminate the possibility of metacharacter-based shell injection (e.g., appending `; rm -rf /` to a filename). The agent must express its intent in valid Python, which is much more constrained and easier to analyze than arbitrary shell scripts.

B. Layer 2: Static Analysis and AST Inspection

Before execution, the generated Python code is parsed into an Abstract Syntax Tree (AST). The system applies a strict security scanner that visits every node of the tree.

- **Module Blacklisting:** Blocks imports of dangerous modules like `os`, `subprocess`, `socket`, and `ctypes`.
- **Function Blacklisting:** Prevents calls to high-risk built-ins such as `eval()`, `exec()`, and `getattr()`.
- **Logic Inspection:** Detects patterns typical of malicious activity, such as obfuscated code or attempts to re-import blocked modules via `importlib`.

C. Layer 3: Path Guardrails and Resource Limits

To prevent the agent from wandering outside its intended workspace, we implement **Path Guardrails**. All file-related tools are wrapped in a validator that checks every path against an "Allowed Paths" whitelist defined in the configuration. Furthermore, we apply **OS-level Resource Limits** (`rlimit`) to the Python process:

- **Memory:** Capped at 1GB (RLIMIT_AS).
- **CPU Time:** 300s timeout per execution.
- **File Size:** Prevents the agent from creating massive log files.

D. Layer 4: Entropy-Based Secret Leak Prevention

To prevent the exfiltration of credentials, `llm-cli` implements a real-time entropy scanner. It calculates the **Shannon Entropy** of every token in the prompt and response:

$$H(X) = -\sum_{i=1}^{n} P(x_i) \log_b P(x_i) \tag{1}$$

Randomly generated strings like API keys exhibit high entropy compared to natural language. Our scanner identifies these "high-entropy anomalies" mid-stream. Unlike traditional regex-based detection, this approach identifies previously unknown secrets (zero-day leaks) without high false-positive rates by differentiating between natural language and high-randomness character sets.

E. Layer 5: Linux Sandboxing (Bubblewrap)

The final layer of containment is provided by **Bubblewrap** (`bwrap`). Each Python execution is isolated in a private, unprivileged namespace. This ensures:

- A private, empty `/tmp` and `/home`.
- Read-only mounts for system libraries (`/usr`, `/lib`).
- Disabled network access (unless explicitly permitted for web-search tools).
- No visibility of host processes or sensitive environment variables.

V. Evaluation

We evaluated the guardrail system against a benchmark of 50 adversarial prompts designed to trigger shell-injection, unauthorized file access, or secret exfiltration.

A. Performance Latency (Empirical)

Experiments were conducted on an **AMD Ryzen 5 8540U** system. Table I summarizes the overhead.

TABLE I
Guardrail Performance Metrics

Metric	Latency (ms)	Overhead Ratio
Secret Detection (Entropy Scan)	0.063	¡ 0.01%
AST Static Analysis	0.42	0.03%
Base Python Execution	13.77	1.15%
Sandboxed Execution (Bubblewrap)	19.40	1.62%
Total Containment Overhead	**6.05**	**0.51%**

B. Containment Effectiveness

The "No-Shell" architecture and AST inspection successfully blocked 100% of attempted injection vectors. The sandboxing layer prevented access to system-critical files (e.g., `/etc/passwd`) even when "jailbroken" prompts were used. The entropy scanner detected 94% of simulated API key leaks, with a false positive rate of less than 0.1% for standard English text. The combined performance cost (**6 ms**) is negligible compared to average LLM inference times (1-5 seconds).

VI. Conclusion

Securing autonomous agents requires a departure from traditional perimeter-based security toward internal, structural guardrails. By adopting a "No-Shell" architecture, high-performance sandboxing, and real-time entropy analysis, the `llm-cli` framework establishes a robust foundation for safe tool use. This paves the way for Phase 2, where we introduce behavioral monitoring to detect semantic-level anomalies that bypass structural checks.

References

[1] Anthropic, "Model Context Protocol (MCP) Specification," 2024.

[2] Y. Ando, "Zero-Trust Architecture for MCP-Based AI Agents: Continuous Identity and Behavioral Monitoring," *TechRxiv*, 2026.

[3] Y. Ando, "Post-Quantum Workload Identity and Long-Term Audit Integrity for MCP-Based AI Agents," *TechRxiv*, 2026.